

SISTEMAS
DISTRIBUIDOS

Practica 1.

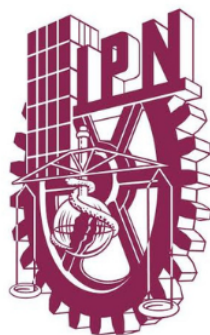
Multiprocesamiento con hilos.

MICHELLE ANGUIANO RODEA

ESCOM

PROFESOR CHADWICK CARRETO
ARELLANO

GRUPO: 7CM6



Instituto
politécnico
nacional

ESCOM

1. Antecedentes (Introducción)

En los sistemas operativos modernos, la **conurrencia** es un concepto fundamental que permite a un programa realizar múltiples tareas simultáneamente. Esto no se logra ejecutando varias copias del programa, sino utilizando **hilos** (threads), que son las unidades de ejecución más pequeñas que un sistema operativo puede gestionar. Cada hilo comparte los recursos del proceso padre (como la memoria), pero tiene su propio flujo de ejecución.

La gestión del ciclo de vida de un hilo es crucial. Un hilo no está siempre en ejecución; pasa por varios estados, como se muestra en el diagrama de estados de procesos/hilos. Estos estados incluyen:

- **Nuevo (New):** El hilo ha sido creado pero aún no se ha iniciado.
- **Ejecutable (Runnable):** El hilo está listo para ejecutarse y espera su turno en la CPU.
- **En Ejecución (Running):** El hilo está actualmente utilizando la CPU.
- **Bloqueado/En Espera (Blocked/Waiting):** El hilo está detenido, esperando que ocurra un evento externo, como la liberación de un recurso, la finalización de una operación de E/S, o una notificación de otro hilo.
- **Terminado (Terminated):** El hilo ha completado su ejecución.

Comprender y simular estos estados es esencial para desarrollar aplicaciones robustas y eficientes que manejen tareas complejas de manera concurrente.

2. Problemática

El objetivo de esta práctica era desarrollar una aplicación en Java que simulara y demostrara visiblemente los diferentes estados del ciclo de vida de un hilo. El principal desafío en la programación concurrente no es solo iniciar hilos, sino gestionar su **interacción y sincronización** de manera segura.

Al intentar implementar la interacción entre dos hilos —uno "esperador" y otro "notificador"— surgió un problema crítico conocido como **condición de carrera** (*race condition*). El programa entraba en un **bloqueo indefinido** (*deadlock*) donde el hilo esperador se quedaba en el estado WAITING para siempre.

Esto ocurría porque el hilo notificador podía ejecutar y enviar su notificación (`notify()`) *antes* de que el hilo esperador tuviera tiempo de ponerse a escuchar (`wait()`). Como la notificación no se "guarda", se perdía, y el hilo esperador, al ejecutarse más tarde, se quedaba esperando una señal que nunca llegaría, deteniendo el flujo del programa.

3. Solución

La solución a la "notificación perdida" y a la condición de carrera fue implementar un mecanismo de sincronización más robusto utilizando una **variable de condición** (también conocida como bandera o *flag*).

La lógica implementada fue la siguiente:

- **Crear una bandera compartida:** Se introdujo una variable booleana `hiloListoParaSerNotificado` en el objeto `RecursoCompartido`. Esta bandera actúa como una "memoria" que indica el estado de la interacción.
- **Proteger el acceso:** Tanto el hilo que espera como el que notifica deben adquirir un bloqueo (`synchronized`) sobre el recurso compartido antes de leer o modificar la bandera.
- **Espera segura (Guarded Block):** El hilo esperador, en lugar de llamar a `wait()` incondicionalmente, lo hace dentro de un bucle `while`. Este bucle comprueba el estado de la bandera: mientras la condición para despertar no sea verdadera (`!hiloListoParaSerNotificado`), el hilo se duerme con `wait()`. Si despierta, vuelve a comprobar la bandera antes de continuar. Esto lo protege tanto de notificaciones perdidas como de "despertares espurios" (*spurious wakeups*).
- **Notificación segura:** El hilo notificador primero establece la bandera en `true` y *después* llama a `notify()`. De esta manera, si el notificador se adelanta, deja la "puerta abierta" para que el esperador, cuando llegue, vea la bandera y continúe sin tener que esperar.

Esta técnica asegura que la comunicación entre los hilos sea fiable, sin importar el orden en que el planificador del sistema operativo decida ejecutarlos.

4. Implementación

El proyecto se implementó en Java utilizando tres clases:

- **RecursoCompartido.java:** Encapsula la lógica de sincronización. Contiene los métodos `synchronized` y la bandera booleana que previene la condición de carrera.
 - `metodoSincronizado()`: Demuestra el estado `BLOCKED`.
 - `esperar()`: Implementa el bucle de espera segura (`while (!flag) { wait(); }`).
 - `notificar()`: Cambia la bandera y notifica al otro hilo.

```

public class RecursoCompartido {

    private boolean hiloListoParaSerNotificado = false;

    // Método para demostrar el estado BLOCKED
    public synchronized void metodoSincronizado(String nombreHilo) {
        System.out.println("-> " + nombreHilo + " ha entrado al método sincronizado.");
        try {
            Thread.sleep(millis:2000); // Simula trabajo
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        System.out.println("<- " + nombreHilo + " está saliendo del método sincronizado.");
    }

    // Método para que un hilo espere de forma segura
    public synchronized void esperar() {
        while (!hiloListoParaSerNotificado) {
            try {
                System.out.println("-> " + Thread.currentThread().getName() + " se prepara para esperar.");
                wait(); // El hilo se pone en WAITING y libera el bloqueo
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        // Una vez que es notificado y la condición es verdadera, el bucle termina.
        System.out.println("<- " + Thread.currentThread().getName() + " ha sido notificado y continúa.");
        hiloListoParaSerNotificado = false;
    }

    public synchronized void notificar() {
        System.out.println("📢 " + Thread.currentThread().getName() + " está cambiando la condición y va a notificar");
        // 1. Cambia la condición (La "memoria" compartida)
        hiloListoParaSerNotificado = true;
        // 2. Notifica a un hilo que esté esperando por este objeto.
        // Si nadie está esperando aún, la bandera 'true' asegurará que no esperen cuando lleguen.
        notify();
    }
}

```

- **HiloTrabajador.java:** Define el comportamiento de los hilos. Su método run() simula las transiciones de estado:
 1. Entra en un método sincronizado para demostrar el bloqueo.
 2. Llama a Thread.sleep() para pasar al estado TIMED_WAITING (Dormido).
 3. Participa en la lógica de esperar()/notificar() para demostrar los estados WAITING y RUNNABLE.

- **SimuladorEstados.java:** Es la clase principal que:

Crea una instancia del RecursoCompartido.

Crea e inicia dos HiloTrabajador, uno configurado como esperador y otro como notificador.

Monitorea e imprime en consola el estado (thread.getState()) de ambos hilos a intervalos regulares hasta que ambos terminan, permitiendo observar las transiciones en tiempo real.

5. Resultados

Al ejecutar la versión corregida del programa, se obtuvo la salida esperada. La consola mostró claramente las transiciones de estado de ambos hilos:

1. **NEW:** Antes de llamar a `.start()`.
2. **RUNNABLE:** Los hilos inician y compiten por la CPU.
3. **BLOCKED:** Un hilo intenta entrar al método sincronizado mientras el otro está adentro.
4. **TIMED_WAITING:** Ambos hilos pausan su ejecución al llamar a `Thread.sleep()`.
5. **WAITING:** El hilo esperador llama a `wait()` y se detiene.
6. **TERMINATED:** Ambos hilos completan su método `run()` exitosamente y finalizan.

El problema del bloqueo indefinido fue completamente resuelto. Sin importar cuántas veces se ejecutó el programa, el Hilo-A (Esperador) siempre fue notificado correctamente y ambos hilos llegaron a su fin. El monitoreo en tiempo real confirmó que el programa ya no se quedaba atascado con un hilo en `WAITING` y otro en `TERMINATED`.

```
Estado de Hilo-A (Esperador) antes de iniciar: NEW

Iniciando hilos...
-----
Estado de Hilo-A (Esperador): RUNNABLE
Estado de Hilo-B (Notificador): RUNNABLE
-----
? El hilo Hilo-A (Esperador) ha iniciado.
? El hilo Hilo-B (Notificador) ha iniciado.
-> Hilo-A (Esperador) ha entrado al método sincronizado.
-----
Estado de Hilo-A (Esperador): TIMED_WAITING
Estado de Hilo-B (Notificador): BLOCKED
-----
-----
Estado de Hilo-A (Esperador): RUNNABLE
Estado de Hilo-B (Notificador): BLOCKED
-----
<- Hilo-A (Esperador) está saliendo del método sincronizado.
-> Hilo-B (Notificador) ha entrado al método sincronizado.
? El hilo Hilo-A (Esperador) se va a dormir por 1.5 segundos.
-----
Estado de Hilo-A (Esperador): TIMED_WAITING
Estado de Hilo-B (Notificador): TIMED_WAITING
-----
<- Hilo-B (Notificador) está saliendo del método sincronizado.
-----
-> Hilo-A (Esperador) se prepara para esperar.
Estado de Hilo-A (Esperador): WAITING
Estado de Hilo-B (Notificador): TIMED_WAITING
-----
-----
```

```

-----
Estado de Hilo-A (Esperador): RUNNABLE
Estado de Hilo-B (Notificador): BLOCKED
-----
<- Hilo-A (Esperador) está saliendo del método sincronizado.
-> Hilo-B (Notificador) ha entrado al método sincronizado.
? El hilo Hilo-A (Esperador) se va a dormir por 1.5 segundos.
-----
Estado de Hilo-A (Esperador): TIMED_WAITING
Estado de Hilo-B (Notificador): TIMED_WAITING
-----
<- Hilo-B (Notificador) está saliendo del método sincronizado.
-----
-> Hilo-A (Esperador) se prepara para esperar.
Estado de Hilo-A (Esperador): WAITING
Estado de Hilo-B (Notificador): TIMED_WAITING
-----
-----
Estado de Hilo-A (Esperador): WAITING
Estado de Hilo-B (Notificador): TIMED_WAITING
-----
? Hilo-B (Notificador) está cambiando la condición y va a notificar.
? El hilo Hilo-B (Notificador) ha terminado su ejecución.
<- Hilo-A (Esperador) ha sido notificado y continúa.
? El hilo Hilo-A (Esperador) ha terminado su ejecución.

Ambos hilos han terminado.
Estado final de Hilo-A (Esperador): TERMINATED
Estado final de Hilo-B (Notificador): TERMINATED
PS C:\Users\miaro\Documents\Sistemas distribuidos\Practica 1>

```

6. Conclusión

Esta práctica fue un ejercicio revelador sobre los fundamentos y los peligros de la programación concurrente. Se demostró exitosamente cómo simular el ciclo de vida de un hilo en Java, observando sus distintos estados en tiempo real.

La aparición del problema de "notificación perdida" sirvió como una lección práctica invaluable sobre la importancia de una sincronización adecuada. Se concluye que el uso de `wait()` y `notify()` debe ser siempre protegido por una variable de condición y un bucle de espera (guarded block). Dependier del *timing* o del orden de ejecución en programas concurrentes es una fuente segura de errores difíciles de depurar.

La solución implementada no solo corrigió el error, sino que representa un patrón de diseño estándar y robusto para la comunicación entre hilos, asegurando que las aplicaciones concurrentes sean fiables y predecibles.

7. Referencias

Baeldung. (n.d.). A guide to Java concurrency. Baeldung. Recuperado el 9 de septiembre de 2025, de <https://www.baeldung.com/java-concurrency>

Baeldung. (n.d.). A guide to java.util.concurrent.Semaphore. Baeldung. Recuperado el 9 de septiembre de 2025, de <https://www.baeldung.com/java-semaphore>

GeeksforGeeks. (n.d.). Multithreading in Java. GeeksforGeeks. Recuperado el 9 de septiembre de 2025, de <https://www.geeksforgeeks.org/multithreading-in-java/>

Jenkov, J. (n.d.). Java concurrency / multithreading tutorial. Jenkov's Tutorials. Recuperado el 9 de septiembre de 2025, de <http://tutorials.jenkov.com/java-concurrency/index.html>